Midterm Review

CS4824/ECE 4424 GTA

Siyi He

Mar. 5, 2026

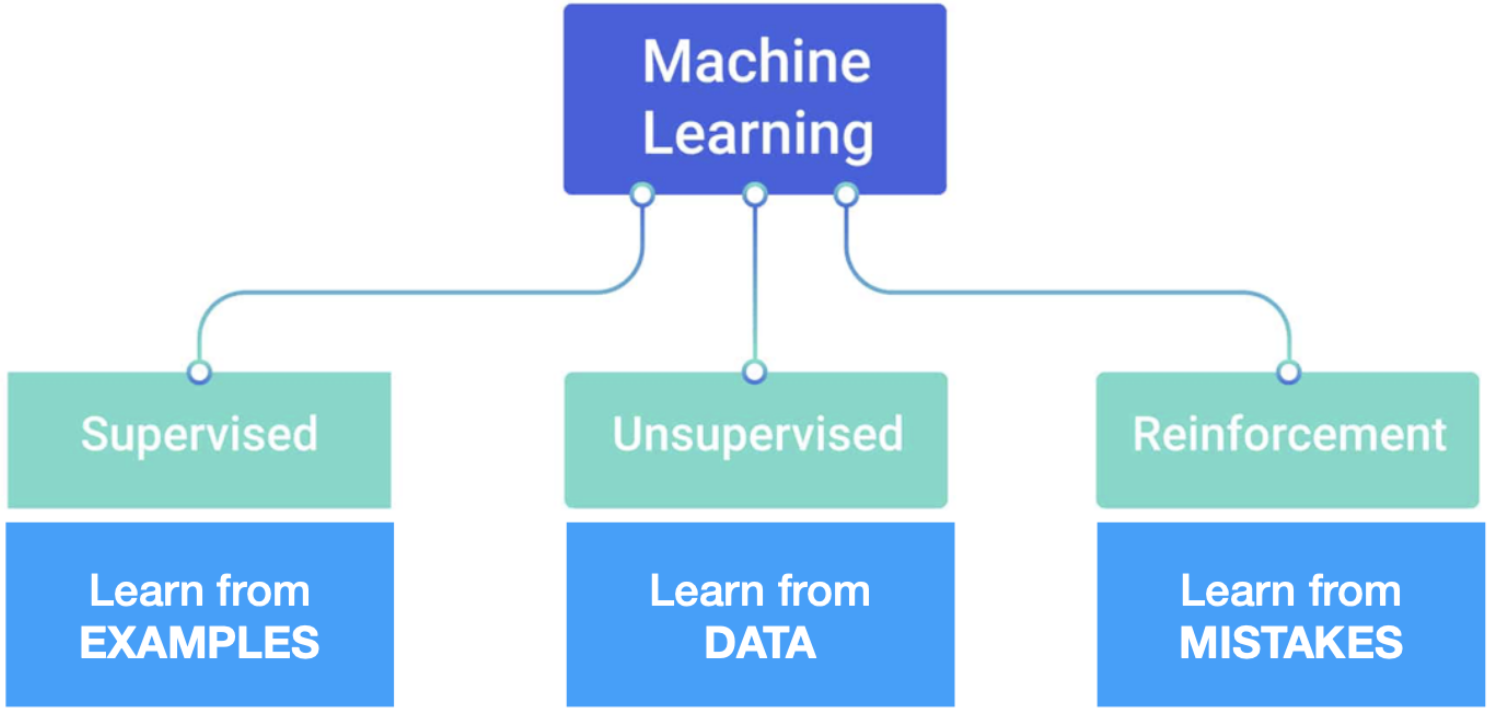
Outline

- Learning Paradigms
- Supervised Learning (Classification & Regression)
- Unsupervised Learning (Clustering & Anomaly Detection)
- Mixture Models
- Learning theory(Bias/variance)

Learning Paradigms

Basic definitions & differences

Learning Paradigms



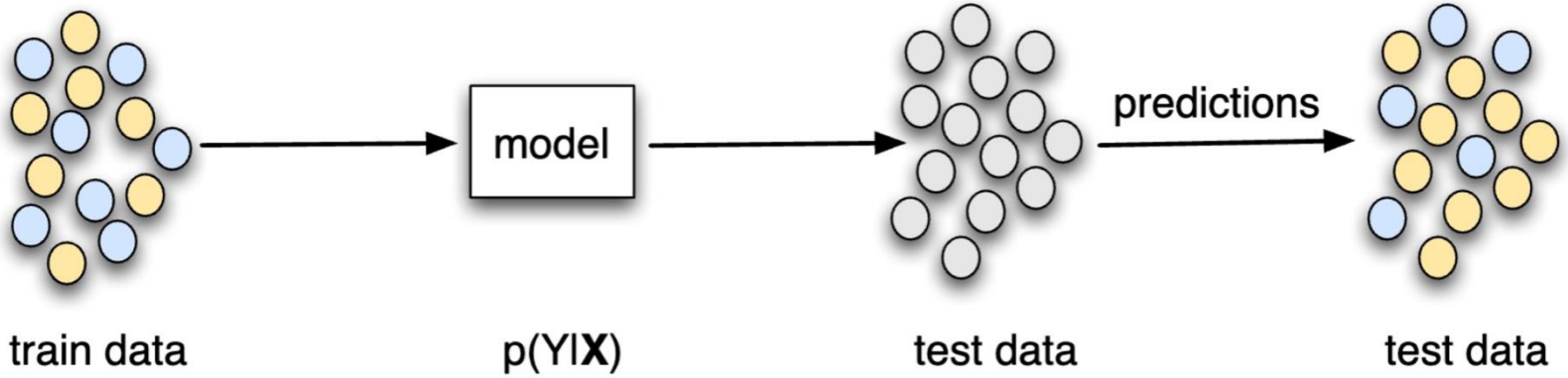
- Input, Output, Goal
- Example

Learning Paradigms - Supervised

- **Input:** Entity – Label pairs $((x_i, y_i))$
- **Goal:** Find predictive relationship between entity & labels
- **Examples:** Classification & Regression

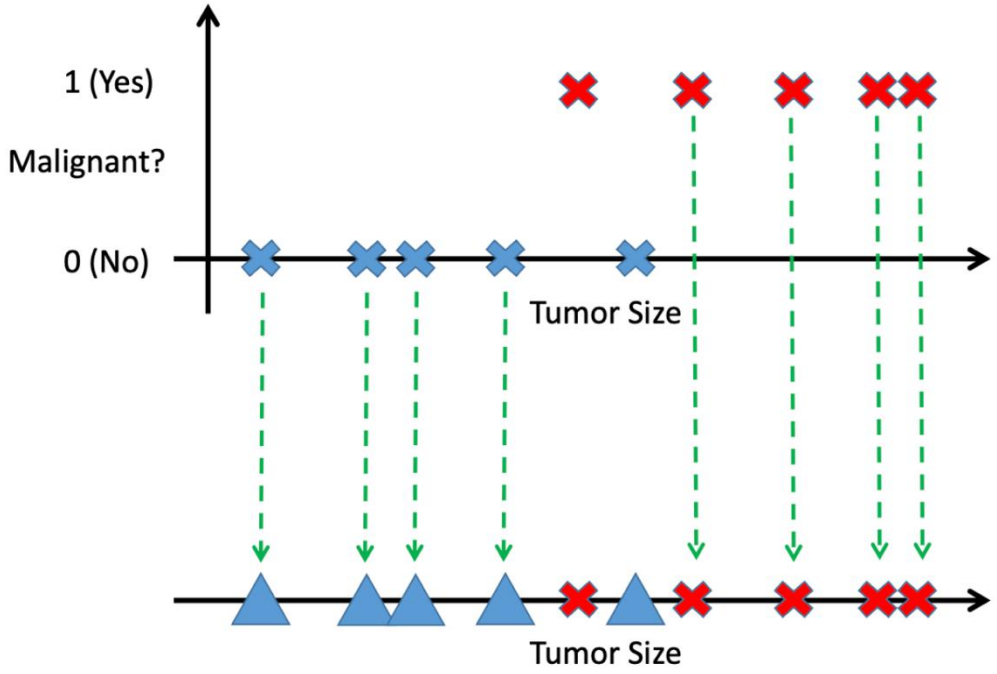
➢Classification: Discrete

➢Regression: Continuous

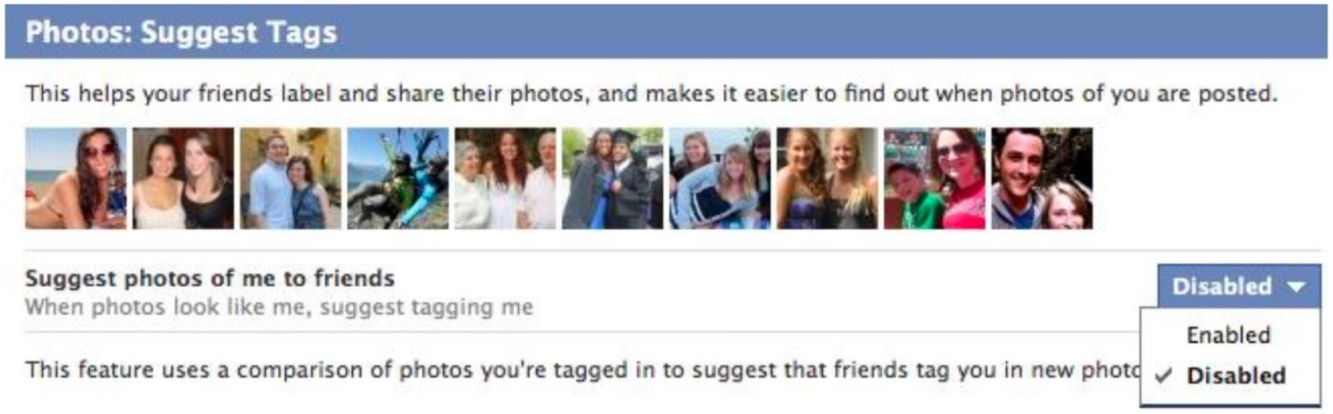


Learning Paradigms - Supervised

- Classification: Binary or Multi-class



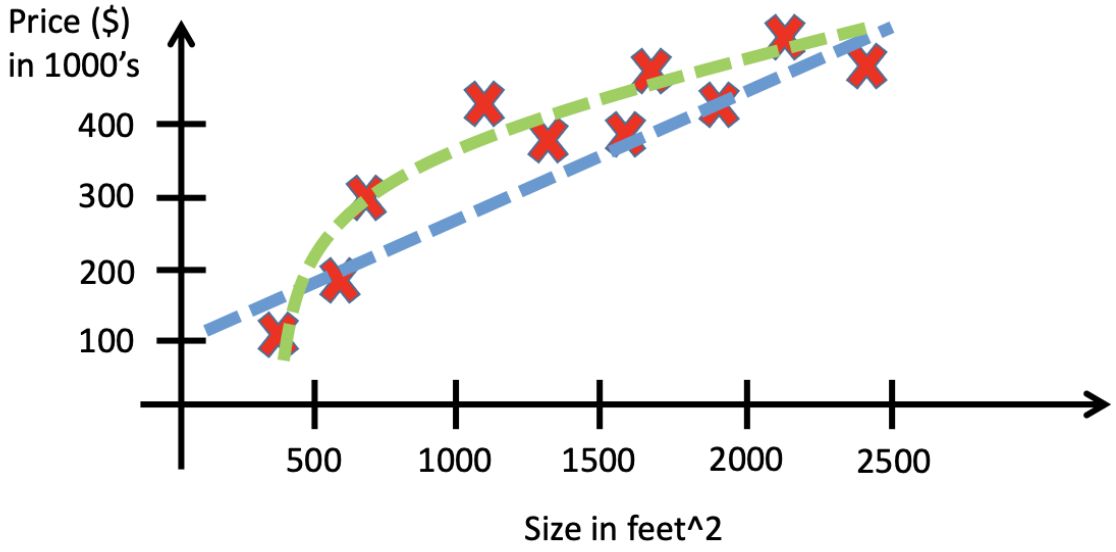
Cancer Diagnosis (M/B, size)



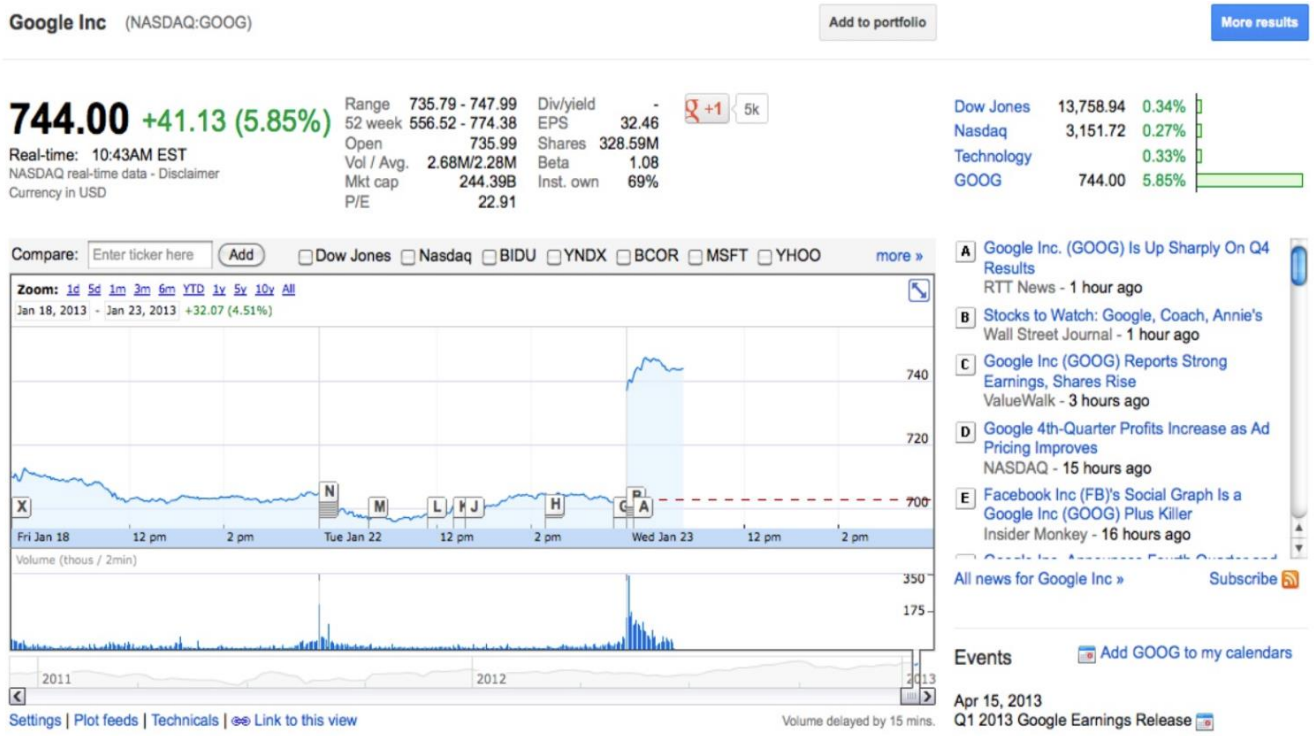
Facebook auto-tagging

Learning Paradigms - Supervised

- Regression:



House price prediction



Stock market

Classification/Regression in-class Exercise

Problem	Task Formulation	Domain	Range	Classification / Regression
Housing valuation 1	predict house price given house size	house size (cont)	price (cont)	Regression
Housing valuation 2	predict whether the house "sells for more or less than the asking price" given house size	house size (cont)	more / less	Binary classification
Age prediction	predict person age based on a given picture	facial features	age range	Multi-class classification
Age prediction	predict person age category (high school, college, graduate) based on a given picture	image features	age category	Multi-class classification
Loan approval prediction	approve or reject loan based on credit score	credit score	approve / reject	Binary classification
Spam detection	classify email into spam / non-spam	keywords	spam / ham	Binary classification
Stock price prediction	Predict stock status: buy, sell or hold	stock values	buy, sell, hold	(time-series) Classification
Speech recognition	convert audio file into text	audio signals	words	Multi-class classification
Digit recognition	digit recognition	images of digits	0 to 9	Multi-class classification



Stork market

Learning Paradigms - Unsupervised

- **Input:** Only entities (x) – **unlabeled** data
- **Goal:** Find interesting structures in data
- **Examples:** Clustering & Anomaly Detection

➢Clustering: groups of similar points

➢Anomaly Detection: recognize the outliers

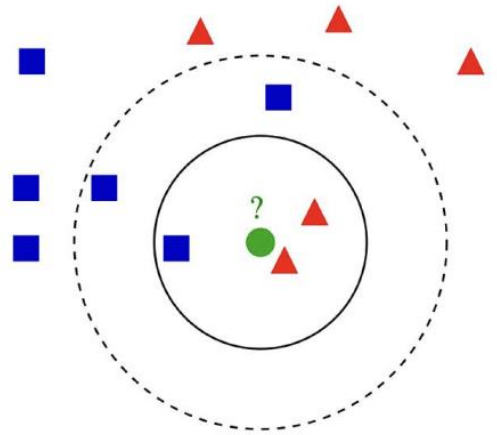
Supervised Learning

Classification (KNN, Bayesian learning – MLE, MAP)

Regression (Linear regression, logistic regression)

Classification

K-Nearest Neighbor



- **Category:** Classification - "You are what you eat"
- **NN classifier:** $h(x) = y_{x^*}$ (a classification function h, the label y, the nearest point to x: x^*)
- $x^* = argmin_{x'} d(x, x')$
- NN to K-NN (K neighbors, not one), voting, most common, $y_x \leftarrow mode(\{y_{x'} | x' \in knn(x)\})$
- **Core:** what K value should we choose?

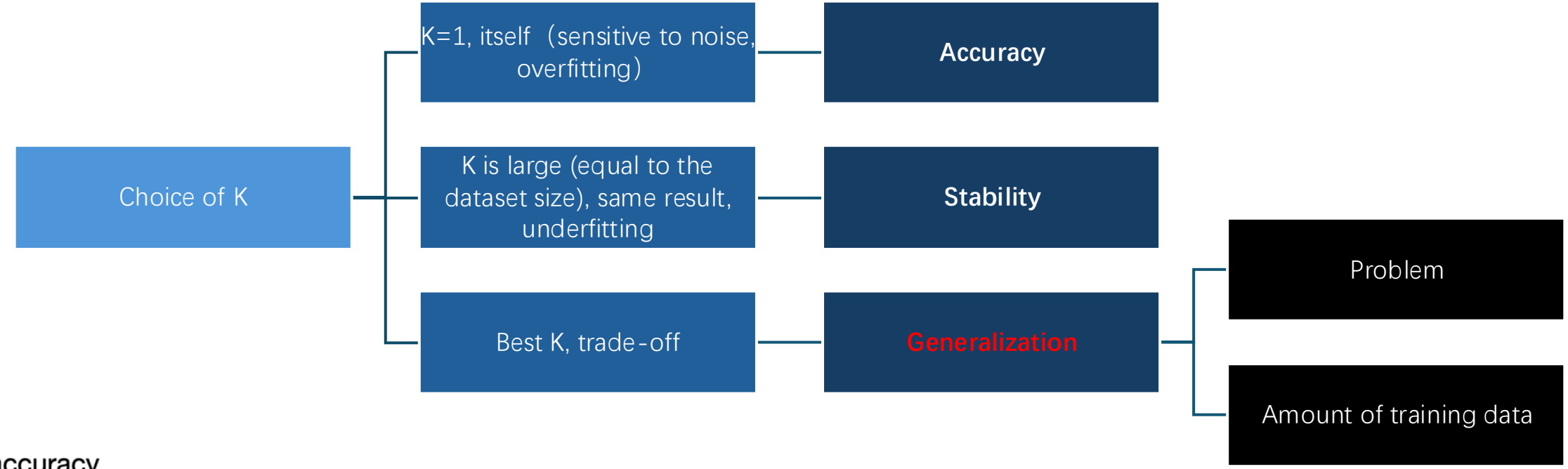
$$L_1: d(x, x') = \sum_j^M |x_j - x'_j|$$

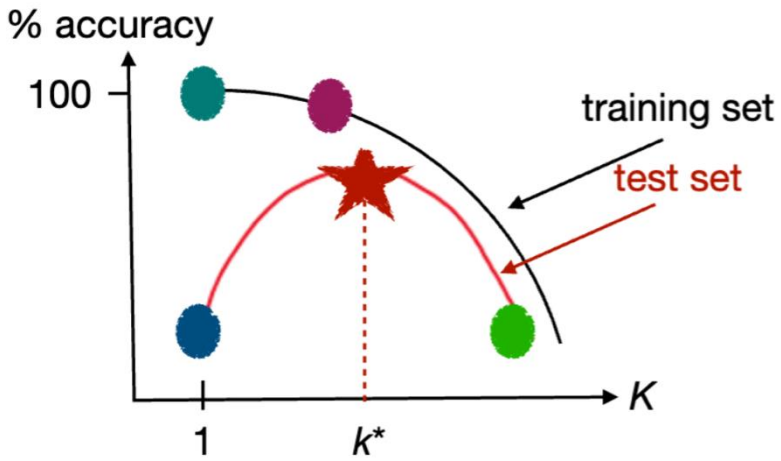
$$L_2: d(x, x') = \left(\sum_j^M |x_j - x'_j|^2\right)^{1/2}$$

...

$$L_p: d(x, x') = \left(\sum_j^M |x_j - x'_j|^p\right)^{1/p}$$

K-Nearest Neighbor – Choice of K



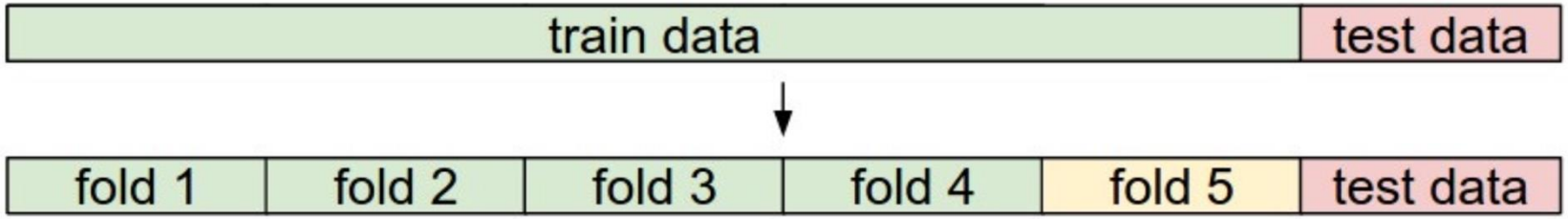


K-Nearest Neighbor – Choice of K

- Future accuracy → Test accuracy (Alternative)
- **No:** Test data as part of training data
- **Approach:** Data splitting

Training set	Validation set	Test set
• Compute nearest neighbor	• Optimize hyperparameters (e.g. K) • A *fake* test set to tune hyperparameter	• Measure Performance

K-Nearest Neighbor – Choice of K



Split dataset → Try different hyperparameters on Validation set → Pick the best → Run it on test set

Bayesian Learning

- **Category:** Classification – Use Bayes rule!

(We classify according to the highest probability given our input data.)

- **Bayes Rule:** Tells the probability of B given that A occurs

Conditional $P(A \mid B) = P(A \wedge B)/P(B)$

Conditional Joint Marginal

Chain $P(A \wedge B) = P(A \mid B)P(B)$

Joint Conditional Marginal

Bayes $P(B \mid A) = [P(A \mid B)P(B)] \;/\; P(A)$

Conditional Conditional Marginals

Posterior Likelihood Prior

$$P(B|A) = \frac{P(A|B)P(B)}{P(A)}$$

Bayesian Learning

sunny

	cold	~cold
headache	0.108	0.012
~headache	0.016	0.064

~sunny

	cold	~cold
headache	0.072	0.008
~headache	0.144	0.576

$$P(headache \wedge cold \mid sunny) = P(headache \wedge cold \wedge sunny)/p(sunny)$$

Conditional probability = 0.108 /(0.108 + 0.012 + 0.016 + 0.064)

= 0.54

$$P(headache \wedge cold \mid \sim sunny) = P(headache \wedge cold \wedge \sim sunny)/p(\sim sunny)$$

= 0.072 / (0.072 + 0.008 + 0.144 + 0.576)

= 0.09

Bayesian Learning – MLE & MAP

Maximum Likelihood Estimation

- Maximize the likelihood of seeing this **evidence(data)** given this **hypothesis (model).**
- $P(e|h)$

Maximum A Posteriori Estimation

- Maximize the posterior of this **hypothesis (model)** being correct given the **evidence (data).**
- $P(h_i|e) \propto P(e|h_i)P(h_i)$

Bayesian Learning – MLE & MAP

	Bayesian	MAP	MLE
	$P(H\|\mathbf{e}) = k\ P(\mathbf{e}\|H)P(H)$ $P(X\|\mathbf{e}) = \sum_i P(X\|h_i)P(h_i\|\mathbf{e})$	$h_{MAP} = argmax_{h_i} P(h_i\|\mathbf{e})$ $P(X\|\mathbf{e}) \approx P(X\|h_{MAP})$	$h_{ML} = argmax_h P(\mathbf{e}\|h)$ $P(X\|\mathbf{e}) \approx P(X\|h_{ML})$
Accuracy	**Optimal**	**Less accurate** than Bayesian	**Less accurate** than Bayesian & MAP
Overfitting	**No overfitting**	**Controlled overfitting**	**Overfitting**
Choosing *h*	All hypotheses are used	Solve optimization to find $\boldsymbol{h_{MAP}}$ (**Intractable**)	• Solve optimization to find $\boldsymbol{h_{ML}}$ **(Intractable)** • Easier (less intractable) than $\boldsymbol{h_{MAP}}$
Making predictions	**Intractable** to compute prediction	Prediction computed using one hypothesis	Prediction computed using one hypothesis

Bayesian Learning - Naïve Bayes

- **Naïve:** Simplified
- **Assumption:** Features are conditionally independent of each other
- **Model:** $h_\theta(x) = P(Y|X_1, X_2, \dots, X_n) \propto P(Y) \prod_i P(X_i|Y)$
- **Cost Function:** (MLE)$J(\theta) = -\log P(\text{Data}|\theta)$

(MAP)$J(\theta) = -\log P(\text{Data}|\theta)\, P(\theta)$

- $\pi_k = P(Y = y_k)$: (Discrete)$\theta_{ijk} = P\left(X_i = x_{ijk} \middle| Y = y_k\right)$

(Continuous)$P(X_i|Y = y_k) = \mathcal{N}\left(X_i \middle| \mu_{ik}, \sigma_{ik}^2\right)$

- **Rule:** $\hat{Y} \leftarrow \arg\max_{y_k} P(Y = y_k) \prod_i P\left(X_i^{test} \middle| Y = y_k\right)$

Bayesian Learning in-class Exercise

$F = 1$ if you live in Fox Ridge

$S = 1$ if you watched the superbowl last night

$D = 1$ if you drive to VT

$G = 1$ if you went to gym in the last month

$P(G = 1|F = 0)$
$= P(G = 1|F = 1)$
$= P(D = 1|F = 0)$
$= P(D = 1|F = 1)$
$= P(S = 1|F = 0)$
$= P(S = 1|F = 1)$
$= P(F = 1) =$

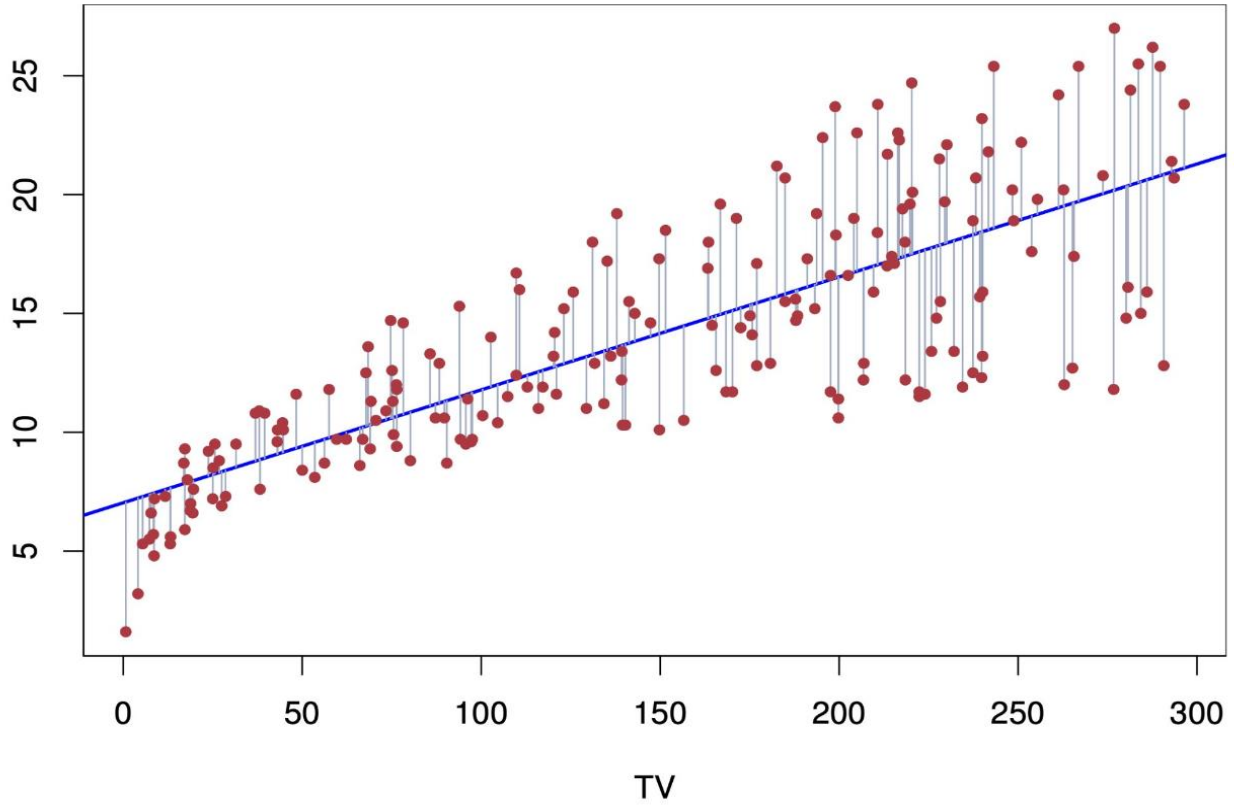
$P(G = 0|F = 0)$
$= P(G = 0|F = 1)$
$= P(D = 0|F = 0)$
$= P(D = 0|F = 1)$
$= P(S = 0|F = 0)$
$= P(S = 0|F = 1)$
$= P(F = 0) =$

$$P(F|S, D, G) \propto P(F)P(S|F)P(D|F)P(G|F)$$

$$P(F|S, D, G) = P(F)P(S|F)P(D|F)P(G|F)$$

Regression

Linear Regression



- **Category:** Regression
- Assume there exists linear relationship between x and y
- **Approach:** Minimize the **error** between predicted y and actual y values (e.g. MSE) – loss function
- **Regularization:** In order to **mitigate overfitting**, we might penalize (regularize) the model for 'bad' weight values. This penalty is added onto the **loss function**. (e.g. ridge penalty $\|w\|_2^2$)
- **Core:** How do we find optimal weights? – Gradient discent

Linear Regression – Gradient Discent

Hypothesis

$$h_\theta(x) = \theta_0 + \theta_1 x$$

Parameters

$$\theta_0, \theta_1$$

Cost Function

$$J(\theta_0, \theta_1) = \frac{1}{2m}\sum_{i=1}^{m}(\hat{y}_i - y_i)^2$$

$$= \frac{1}{2m}\sum_{i=1}^{m}(h_\theta(x_i) - y_i)^2$$

Optimization Objective

$$\min_{\theta_0,\theta_1} J(\theta_0, \theta_1)$$

Outline

1. Start with some θ_0, θ_1
2. Keep changing θ_0, θ_1 to reduce $J(\theta_0, \theta_1)$

Linear Regression – Gradient Descent

- $\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta_0, \theta_1)$

- $\frac{\partial}{\partial \theta_j} J(\theta_0, \theta_1) = \frac{\partial}{\partial \theta_j} \frac{1}{2m} \sum_{i=1}^{m} \left(h_\theta\left(x^{(i)}\right) - y^{(i)} \right)^2$

$$= \frac{\partial}{\partial \theta_j} \frac{1}{2m} \sum_{i=1}^{m} \left(\theta_0 + \theta_1 x^{(i)} - y^{(i)} \right)^2$$

(don't omit process!)

- $j = 0$: $\frac{\partial}{\partial \theta_0} J(\theta_0, \theta_1) = \frac{1}{m} \sum_{i=1}^{m} \left(h_\theta\left(x^{(i)}\right) - y^{(i)} \right)$
- $j = 1$: $\frac{\partial}{\partial \theta_1} J(\theta_0, \theta_1) = \frac{1}{m} \sum_{i=1}^{m} \left(h_\theta\left(x^{(i)}\right) - y^{(i)} \right) x^{(i)}$

Linear Regression – Gradient Descent

- **Regularized form:** $J(\theta) = \frac{1}{2m}\left[\sum_{i=1}^{m}\left(h_\theta\left(x^{(i)}\right) - y^{(i)}\right)^2 + \lambda\sum_{j=1}^{n}\theta_j^2\right]$

Repeat {

$$\theta_0 := \theta_0 - \alpha\frac{1}{m}\sum_{i=1}^{m}\left(h_\theta\left(x^{(i)}\right) - y^{(i)}\right)$$

n : number of features,

θ_0 :not penalized

$$\theta_j := \theta_j - \alpha\frac{1}{m}\left[\sum_{i=1}^{m}\left(h_\theta\left(x^{(i)}\right) - y^{(i)}\right)x_j^{(i)} + \lambda\theta_j\right]$$

}

$$\theta_j := \theta_j\left(1 - \alpha\frac{\lambda}{m}\right) - \alpha\frac{1}{m}\left[\sum_{i=1}^{m}\left(h_\theta\left(x^{(i)}\right) - y^{(i)}\right)x_j^{(i)}\right]$$

Linear Regression in-class Exercise

"squared error function"
"**mean squared error**"

$$J(\theta_0, \theta_1) = \frac{1}{2m}\sum_{i=1}^{m}(\hat{y}_i - y_i)^2 = \frac{1}{2m}\sum_{i=1}^{m}(h_\theta(x_i) - y_i)^2$$

Predicted value

True value

Measure **accuracy of hypothesis function**

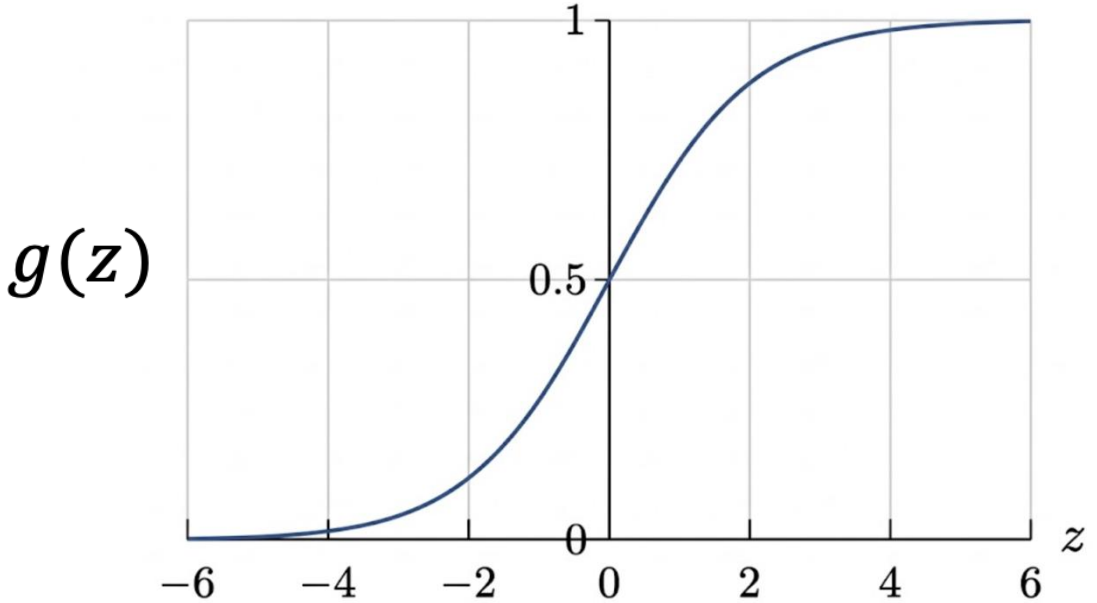
Mean

Error

Square

- What's the cost function of linear regression with L2 regularization?
- What's the partial derivative of θ_0/θ_1? $\frac{\partial}{\partial\theta_0}J(\theta_0, \theta_1), \frac{\partial}{\partial\theta_1}J(\theta_0, \theta_1)$
- How you update the weights of linear regression with L2 regularization?

Logistic Regression



- **Category:** ~~Regression?~~ Classification
- **Approach:** Squish regression output between 0 and 1
- **Key Idea:** Multiply input by weight. If > 0, then classify as 1, else classify as 0.
- **Sigmoid:** $g(z) = \frac{1}{1+e^{-z}}$
- $h_\theta(x) = g(\theta^\top x) = \frac{1}{1+e^{-\theta^\top x}}, 0 \leq h_\theta(x) \leq 1$

Logistic Regression

- $\mathbf{Cost}(h_\theta(x), y) = \begin{cases} -\log(h_\theta(x)) & \text{if } y = 1 \\ -\log(1 - h_\theta(x)) & \text{if } y = 0 \end{cases}$



- $Cost(h_\theta(x), y) = -y \log(h_\theta(x)) - (1 - y) \log(1 - h_\theta(x))$
- Derived from log-likelihood, convex
- If $y = 1$: $\mathbf{Cost}(h_\theta(x), y) = -\log(h_\theta(x))$
- If $y = 0$: $\mathbf{Cost}(h_\theta(x), y) = -\log(1 - h_\theta(x))$

Logistic Regression

Hypothesis

$$h_\theta(x) == \frac{1}{1 + e^{-\theta^\top x}}$$

Parameters

θ

Optimization Objective

$$\min_\theta J(\theta)$$

Cost Function

$$J(\theta) = -\frac{1}{m}\left[\sum_{i=1}^{m} y^{(i)} \log\left(h_\theta\left(x^{(i)}\right)\right) + \left(1 - y^{(i)}\right) \log\left(1 - h_\theta\left(x^{(i)}\right)\right)\right]$$

Analysis

Good News: convex

Bad News: No analytical solution

Outline

Repeat $\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta)$

Simultaneously update all θ_j

Logistic Regression - Comparison

$$\frac{\partial}{\partial \theta_j} J(\theta) = \frac{1}{m} \sum_{i=1}^{m} (h_\theta(x^{(i)}) - y^{(i)}) x_j^{(i)}$$

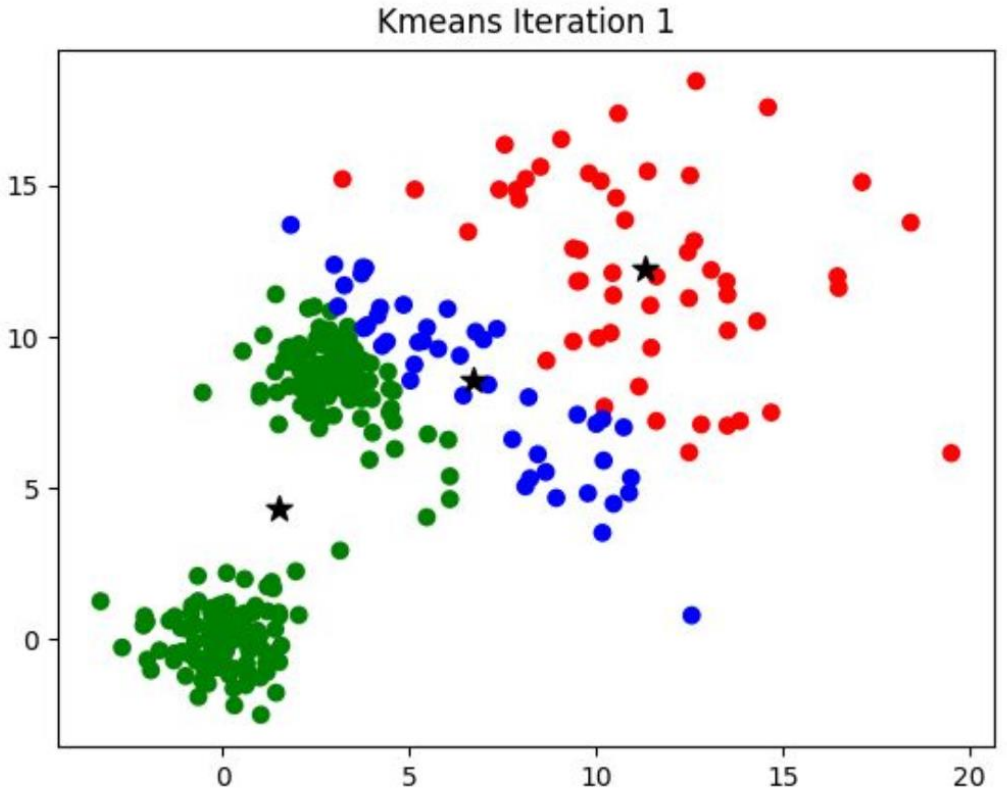
$$\theta_j := \theta_j - \alpha \frac{1}{m} \sum_{i=1}^{m} (h_\theta(x^{(i)}) - y^{(i)}) x_j^{(i)}$$

- For linear regression: $h_\theta(x) = \theta^\top x$
- For logistic regression: $h_\theta(x) = \frac{1}{1+e^{-\theta^\top x}}$

Unsupervised Learning

Clustering & Anomaly Detection

Clustering



- **Goal:** create groups of similar data Points
- **Challenge:** Bad Random Initialization → Local Minimum
- **How does K-means work?**

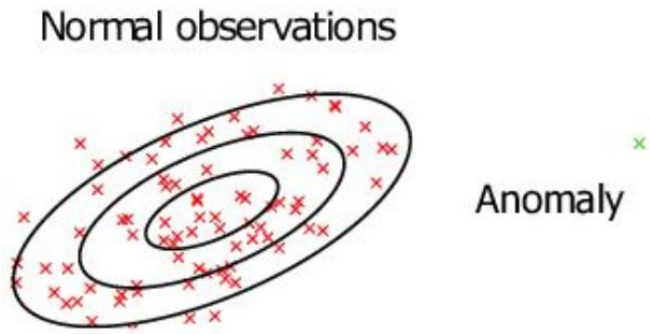
1. Initialize K centroids

2. (1)Cluster Assignment: Assign each data point to the nearest centroid

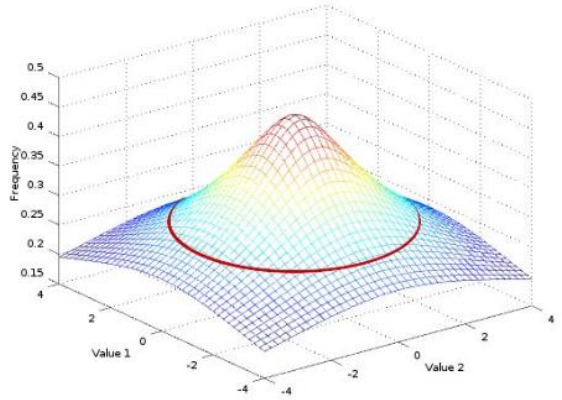
   (2)Compute the mean for each cluster using data assigned to the cluster

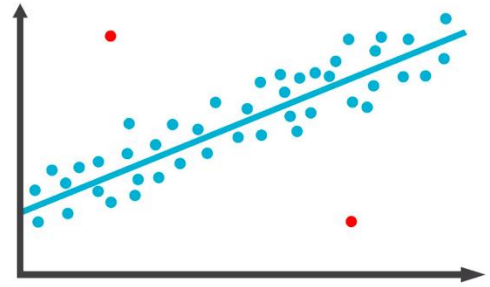
Iteration

Anomaly Detection



- **Goal:** Find outliers – significantly different from other points
- **2 Definitions:**
  - ➢Unusual change over time
  - ➢Blend-in anomaly
- **Tools:**
  - ➢Unusual Change Score (Markov Chain)
  - ➢Score-Level Fusion Weight
  - ➢**How to identify?** Entropy-based (naïve top k & statistical)





Mixture Models

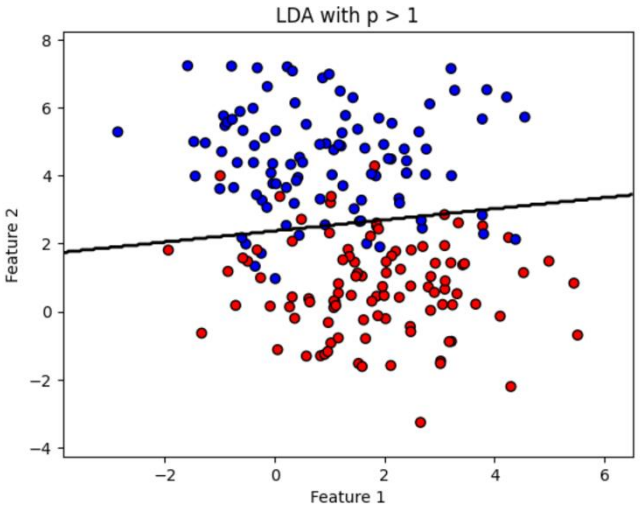
LDA & QDA

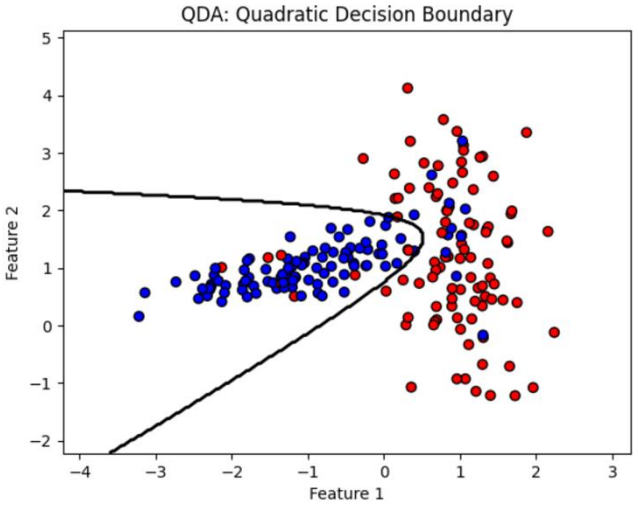
Mixture Models

- MoG (Mixture of Gaussian): Generative Model vs. Discriminative
- Prior(in class K), Likelihood(Gaussian!), Posterior
- **The covariances (covariance matrices Σ) of classes matter!**

➢All Σ are equal: linear decision boundary, LDA

➢All Σ are not equal: quadratic decision boundary, QDA



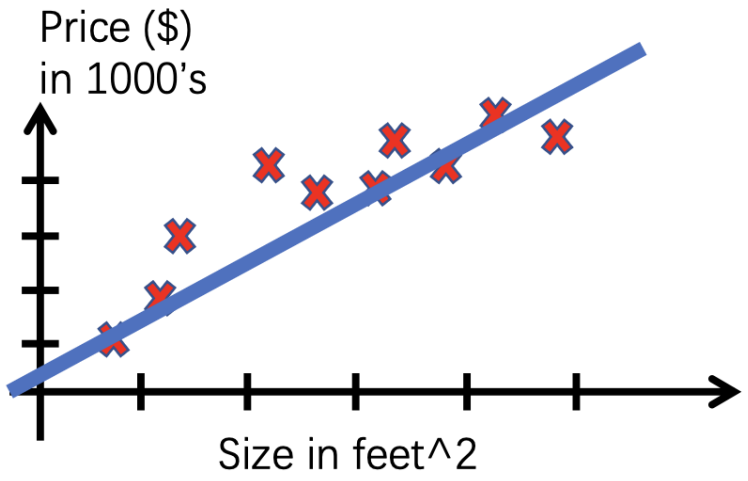


Learning Theory

Bias-Variance tradeoff

Learning Theory (Bias/Variance)

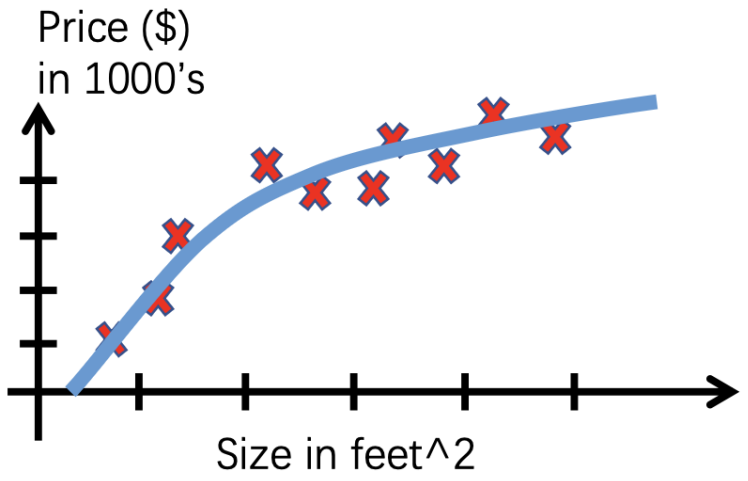
$$J(\theta) = \frac{1}{2m}\sum_{i=1}^{m}\left(h_\theta\left(x^{(i)}\right) - y^{(i)}\right)^2 \approx 0$$



$h_\theta(x) = \theta_0 + \theta_1 x$

High Bias

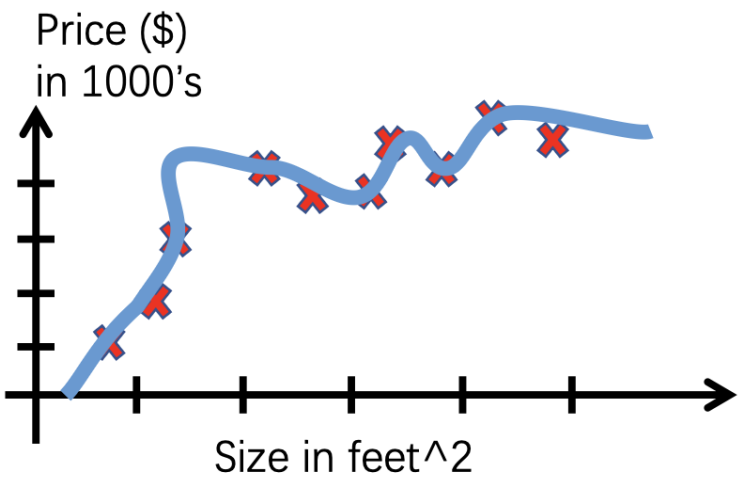
Underfitting



$h_\theta(x) = \theta_0 + \theta_1 x + \theta_2 x^2$

Balance

Just right



$h_\theta(x)$
$= \theta_0 + \theta_1 x + \theta_2 x^2 + \theta_3 x^3$
$+ \theta_4 x^4 + \cdots$

High Variance

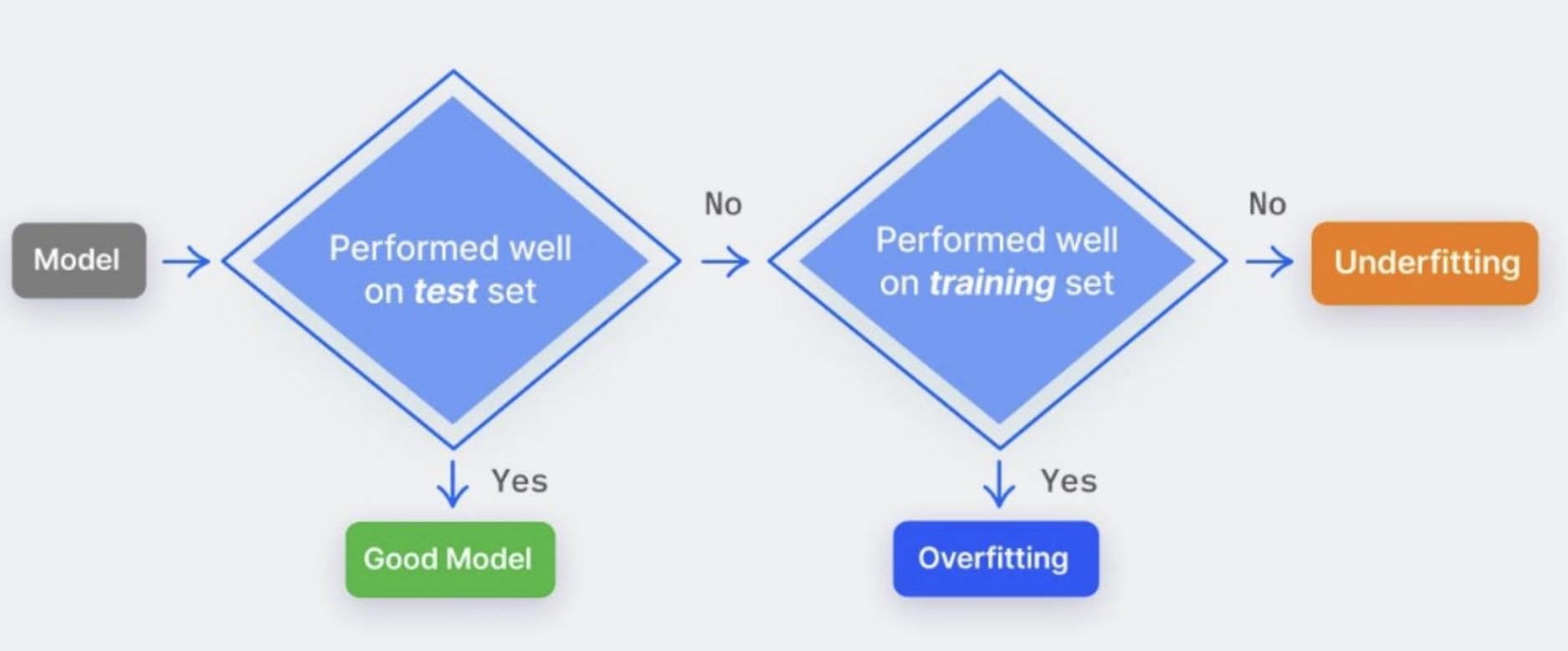
Overfitting

(Too many features (i.e. complex model), fit the training set very well but fail to generalize)

B/V – Bias-Variance Tradeoff

- **Bias:** difference between
  - what you expect to learn and truth
  - Measures how well you expect to represent true solution
  - Decreases with more complex model
- **Variance:** difference between
  - what you expect to learn and what you learn from a particular dataset
  - Measures how sensitive learner is to specific dataset
  - Increases with more complex model

Learning Theory (Bias/Variance)



Learning Theory (Bias/Variance)

How to address overfitting?

- **1. Reduce number of features.**

➢Manually select which features to keep.

➢Model selection algorithm (later in course).

- **2. Regularization.**

➢Keep all the features, but reduce magnitude/values of parameters θ_j.

➢Works well when we have a lot of features, each of which contributes a bit to predicting y.

Good Luck!